ELO320 - Estructuras de Datos y Algoritmos
Ordenamiento en Tiempo Lineal
Counting Sort, Radix Sort

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2023

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Ordenamiento por comparación

InsertSort, MergeSort, QuickSort son algoritmos de ordenamiento basados en comparaciones, vale decir, en el contraste de dos valores.

- Estos algoritmos pueden ser analizados en un arbol binario de decisión, con decisiones $\leq$ y $>$.
- El largo de estos árboles determina una complejidad en función de $\log n$.
- Además, hay al menos n decisiones por los elementos del árbol.
- Por lo tanto, este árbol de decisión actua como conta inferior de todo Ordenamiento por Comparación.
- Ordenamiento por comparación $\rightarrow \Omega(n \log n)$.

Ordenamiento en tiempo lineal

Existen algoritmos de ordenamiento que no depende de comparaciones, y en este curso veremos dos que mantienen complejidad en tiempo lineal:

- Counting Sort.
- Radix Sort.

Ambos son de complejidad lineal, vale decir, $\mathcal{O}(n)$. Sin embargo, se es más fino respecto a cada uno de ellos.

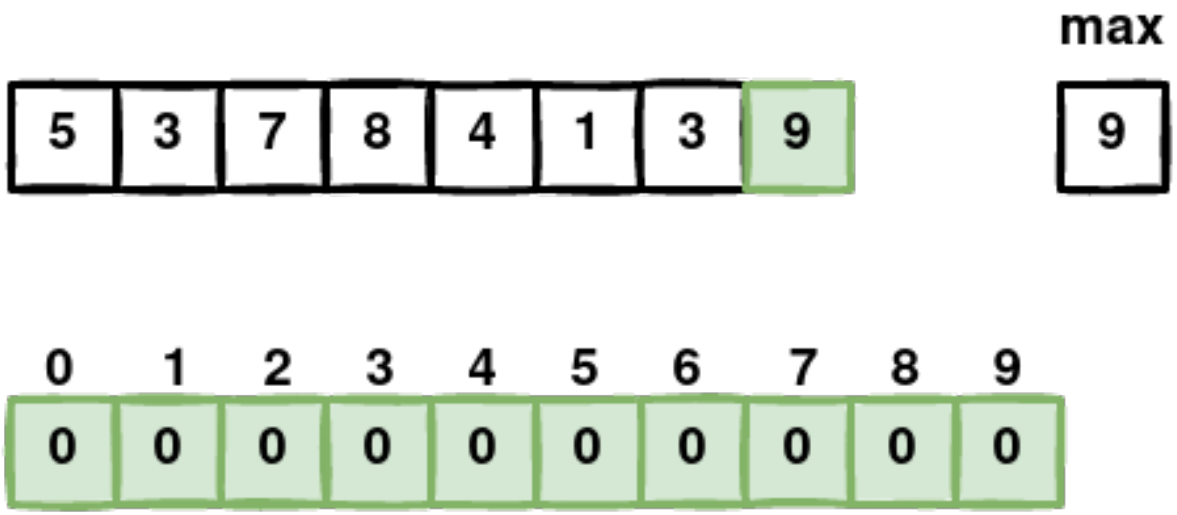
Además, ambos presentan ciertas limitantes para poder ser aplicados.

Counting Sort

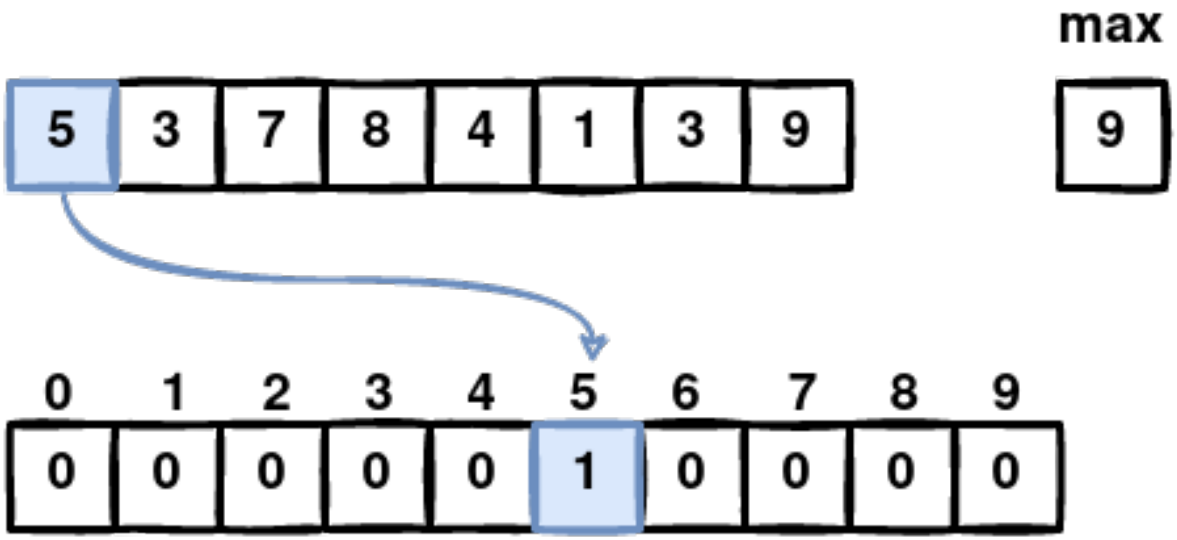
Algoritmo de Ordenamiento basado en contar la cantidad de ocurrencias de los elementos en un arreglo.

- Complejidad: $\mathcal{O}(n + k)$, con n numero de elementos del arreglo y k el máximo elemento del arreglo.
- Uso: enteros y representaciones discretas.
- No se recomienda para strings, floats, etc.
- Es estable.
- Utiliza bastante memoria extra.

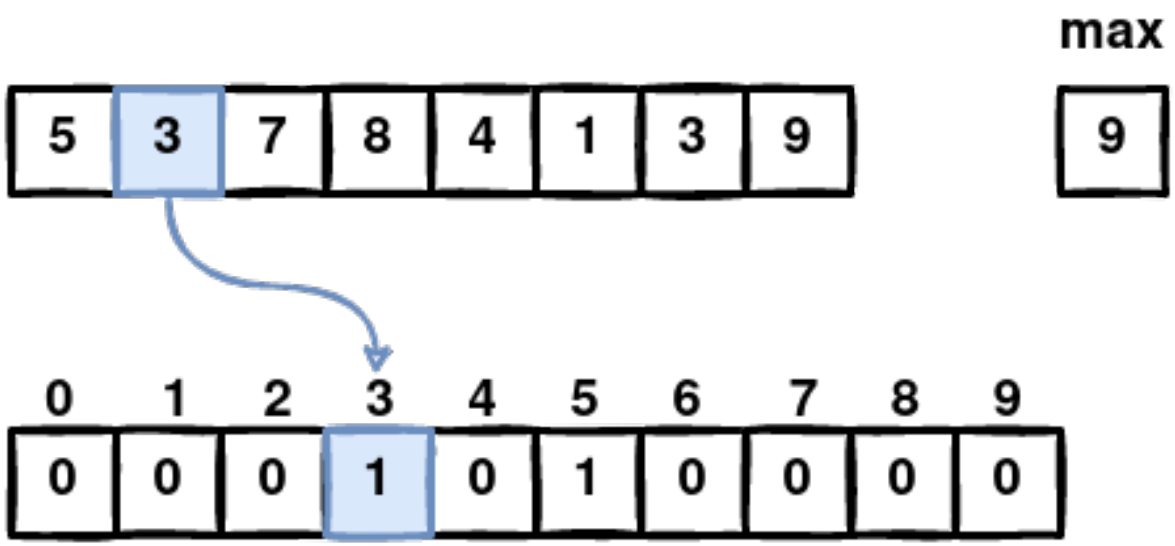
Counting Sort: Idea General



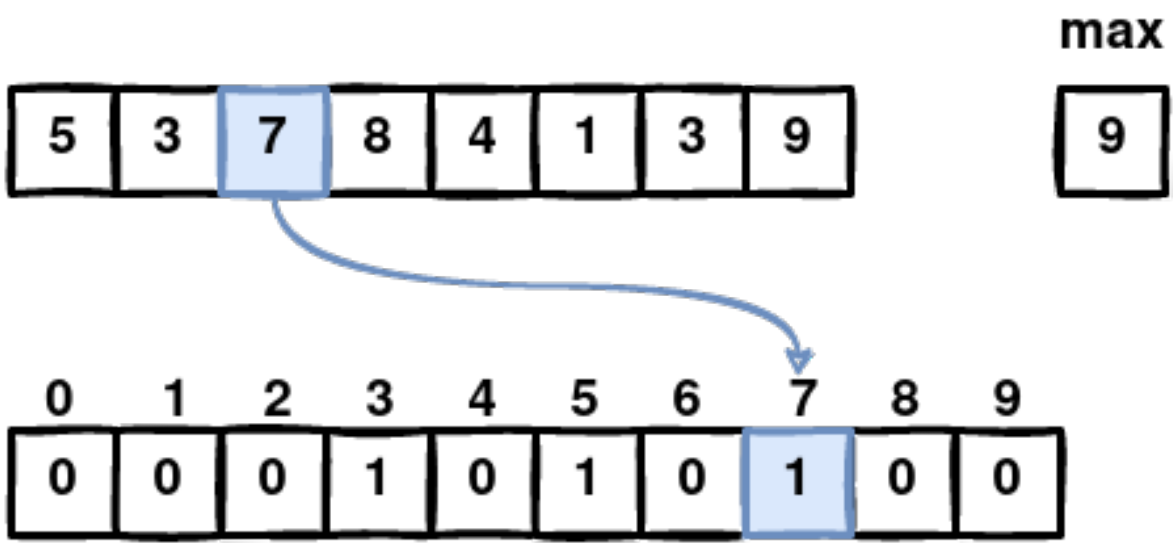
Counting Sort: Idea General



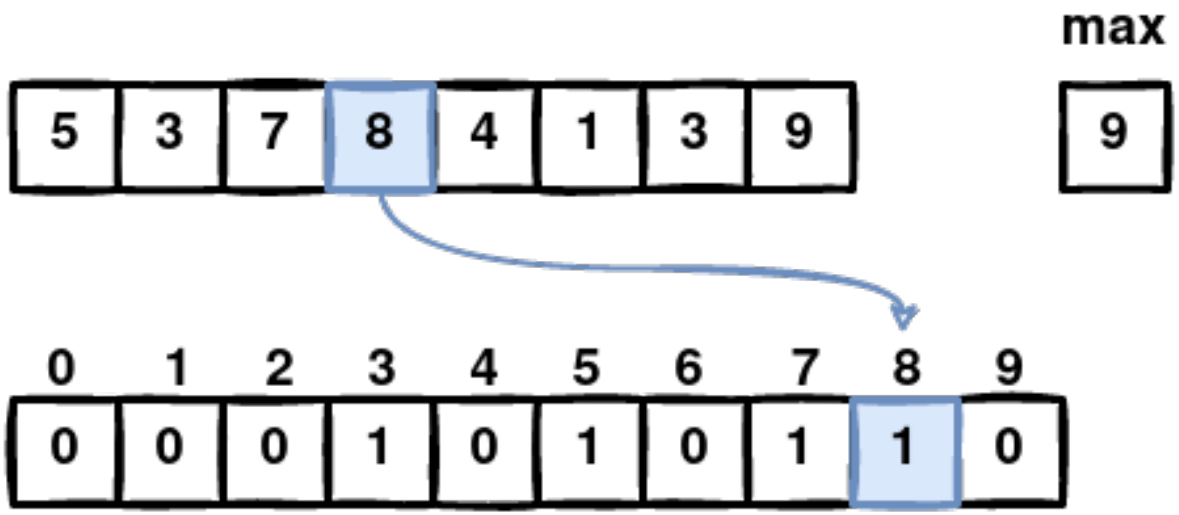
Counting Sort: Idea General



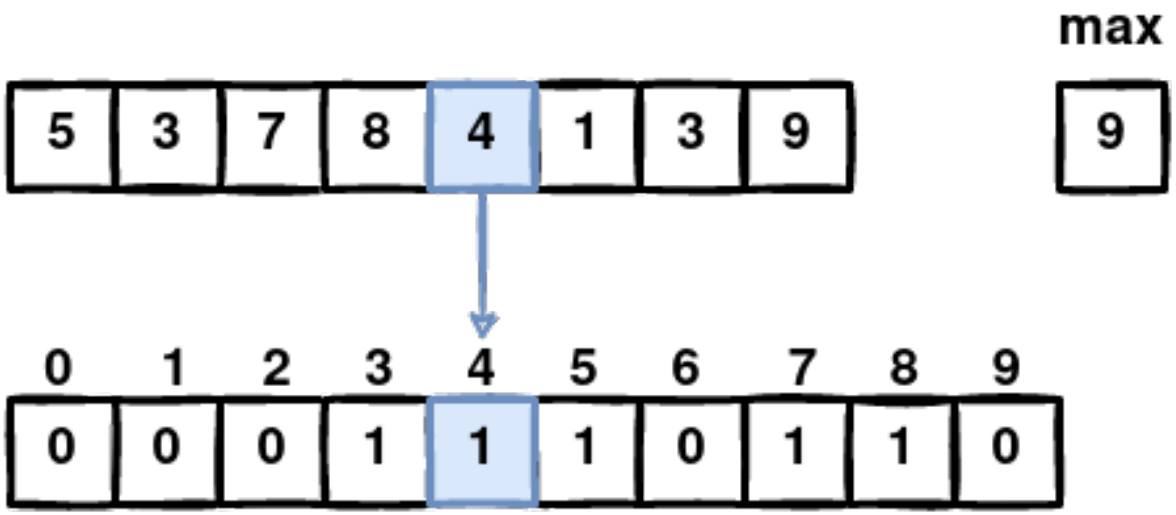
Counting Sort: Idea General



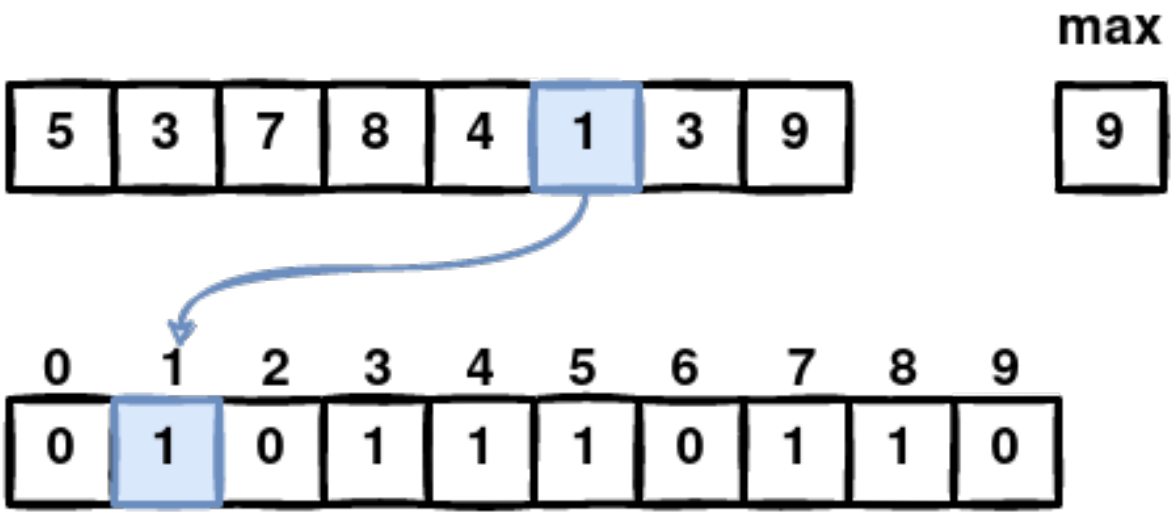
Counting Sort: Idea General



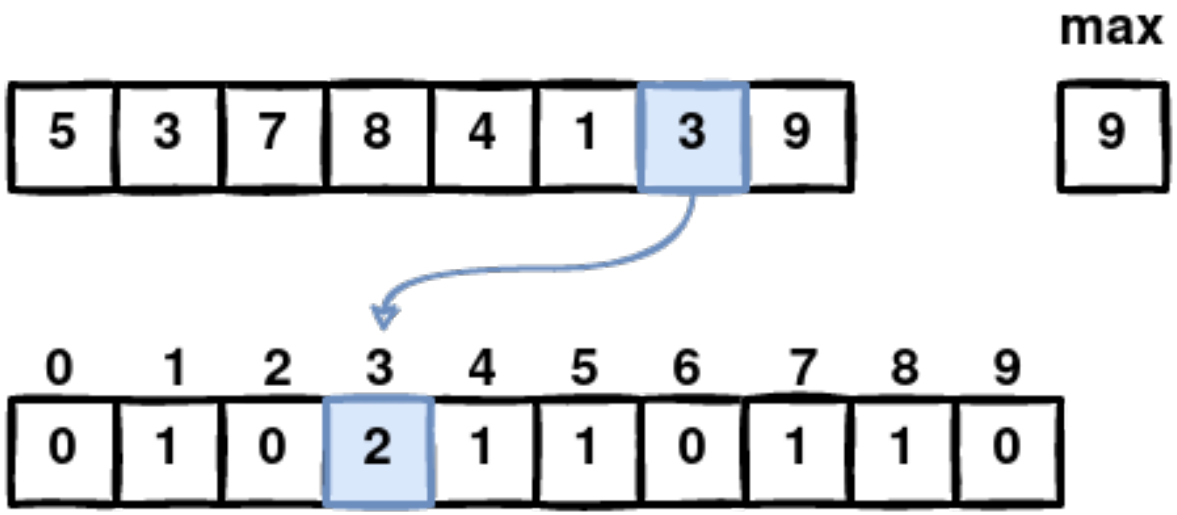
Counting Sort: Idea General



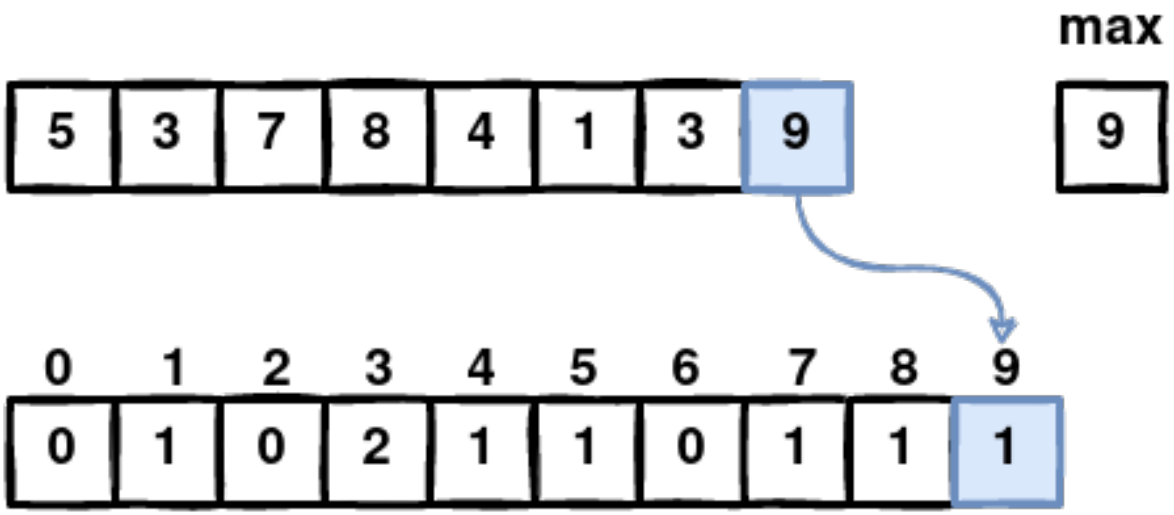
Counting Sort: Idea General



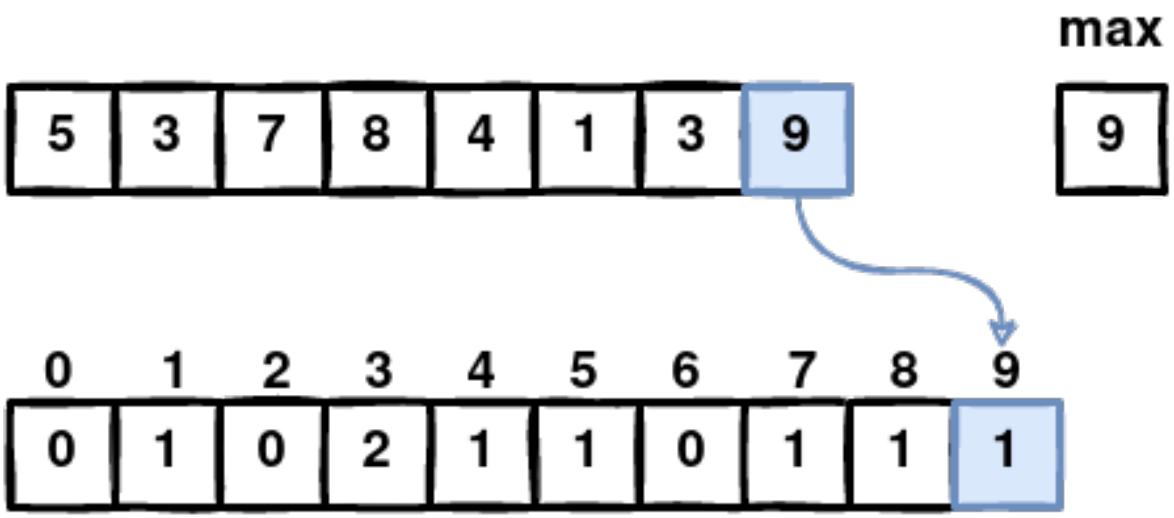
Counting Sort: Idea General



Counting Sort: Idea General

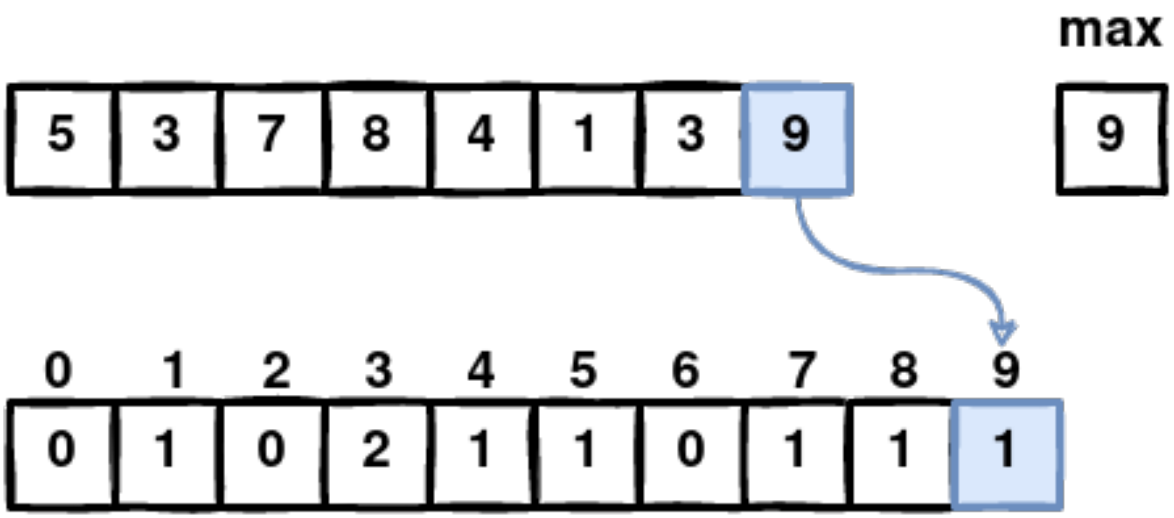


Counting Sort: Idea General



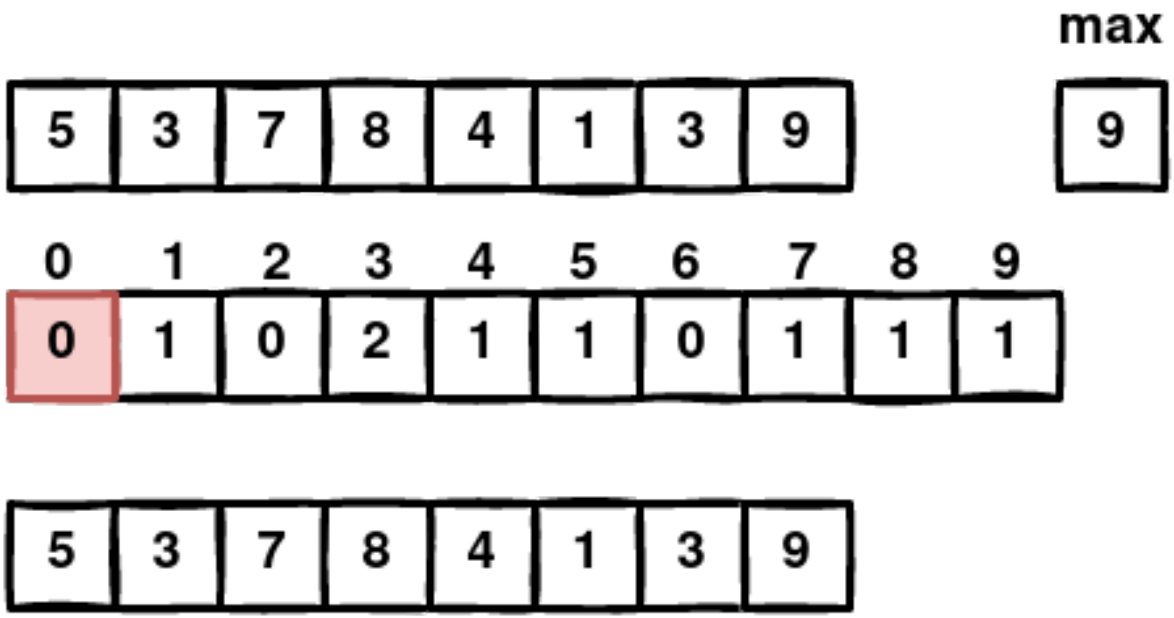
Con esto debemos ser capaces de ordenar. Pause: pero tiene sus inconvenientes.

Counting Sort: Idea General

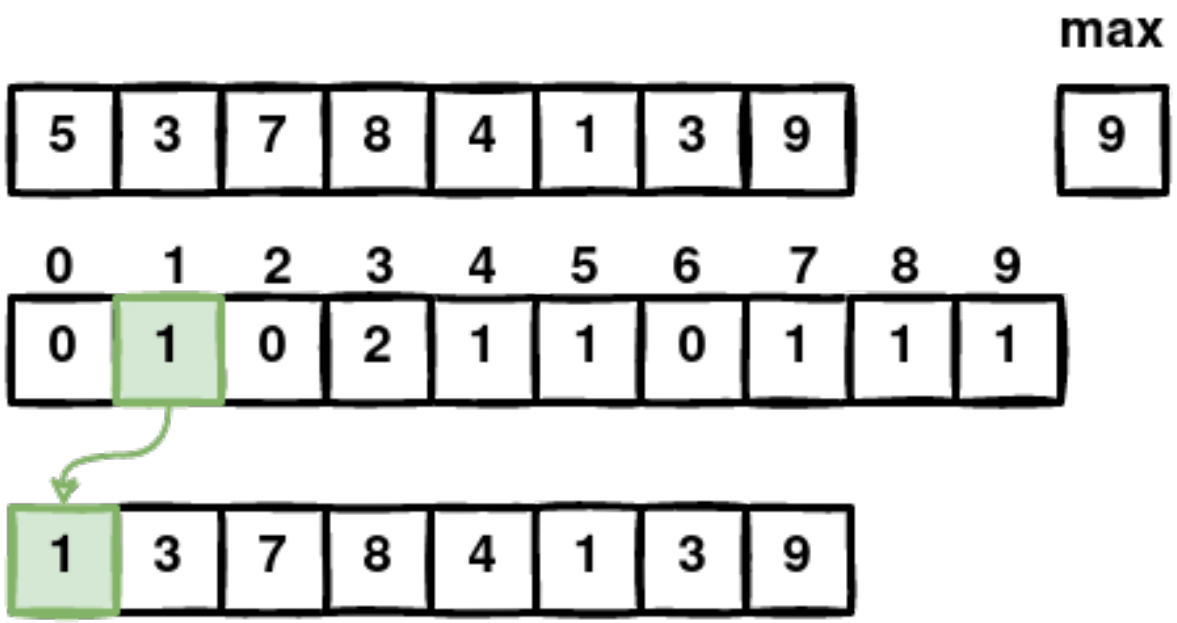


Con esto debemos ser capaces de ordenar. **Pause: pero tiene sus inconvenientes**.

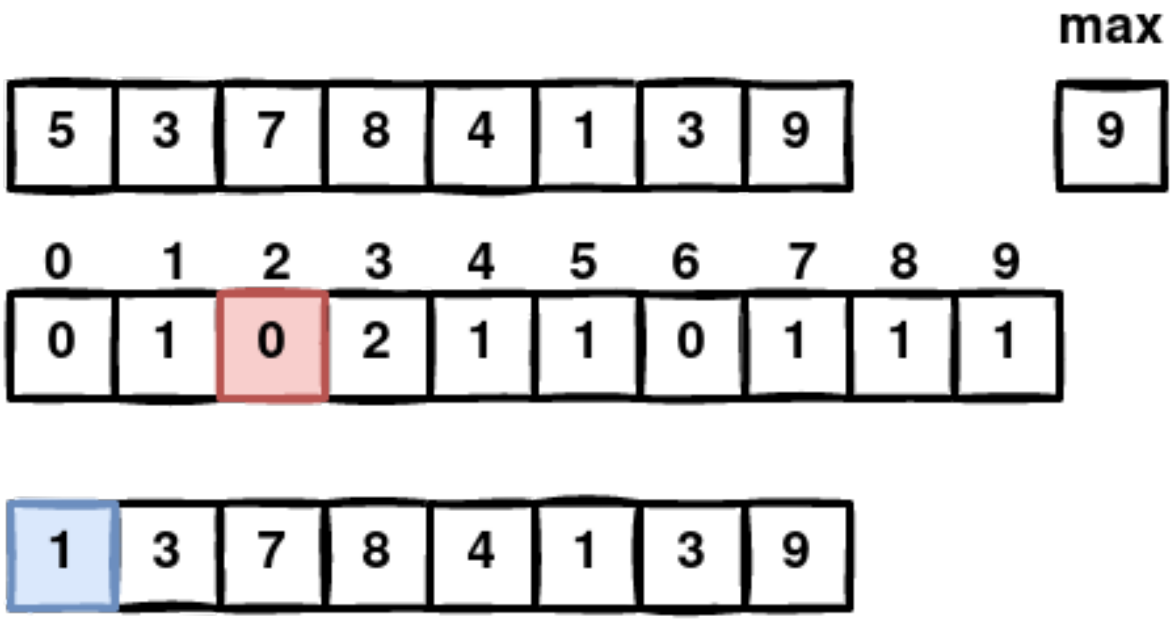
Counting Sort: Idea General



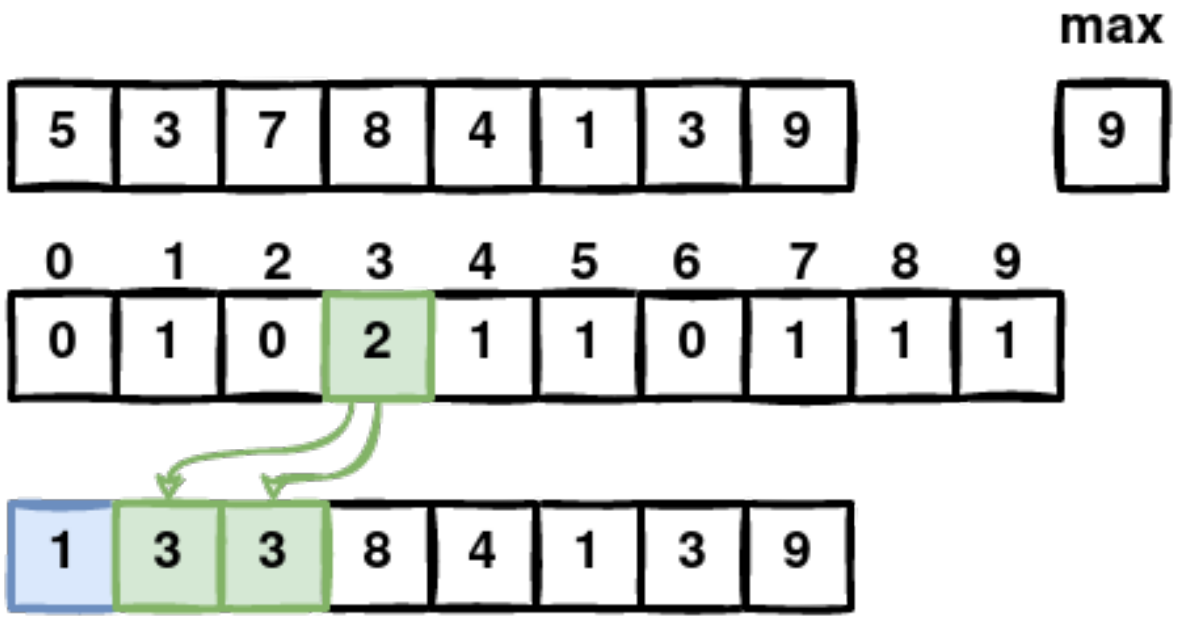
Counting Sort: Idea General



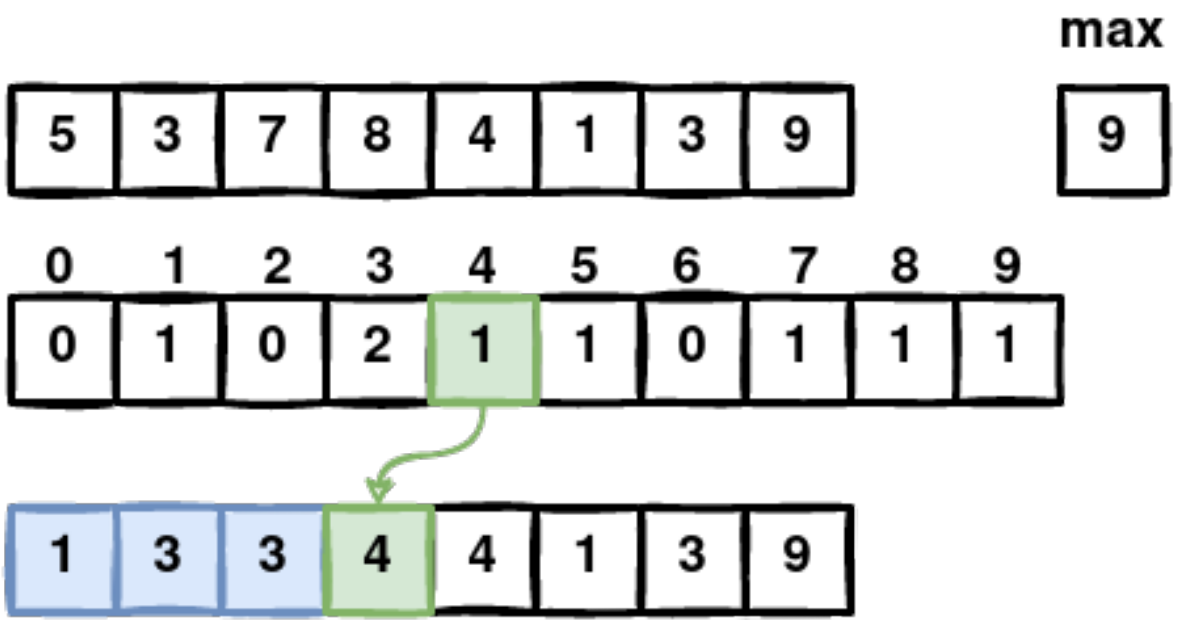
Counting Sort: Idea General



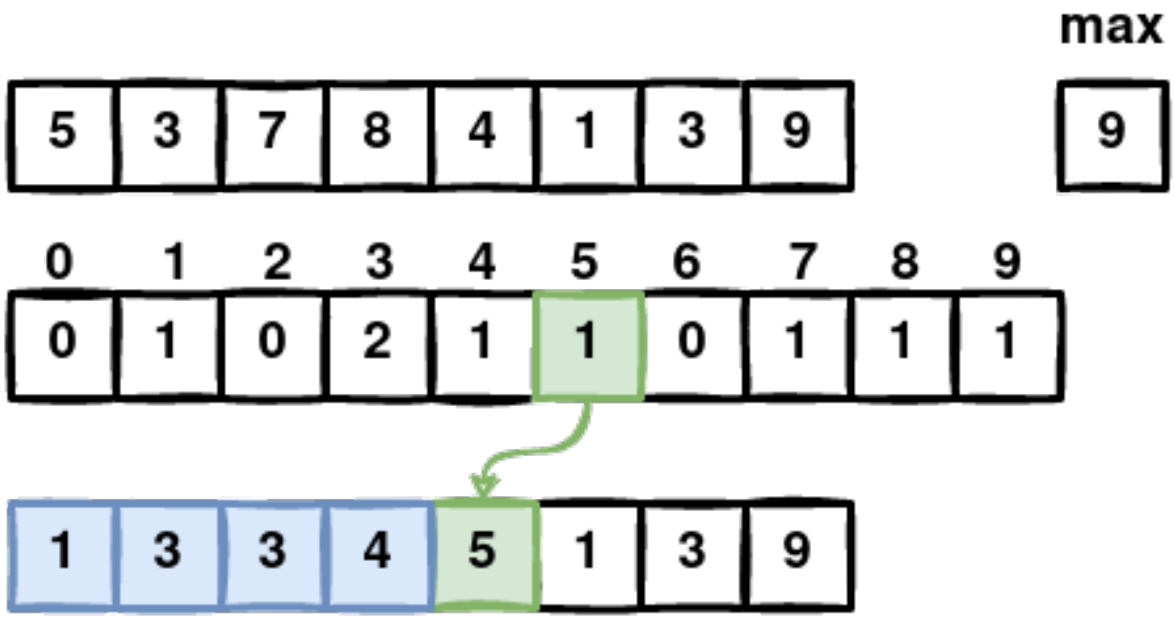
Counting Sort: Idea General



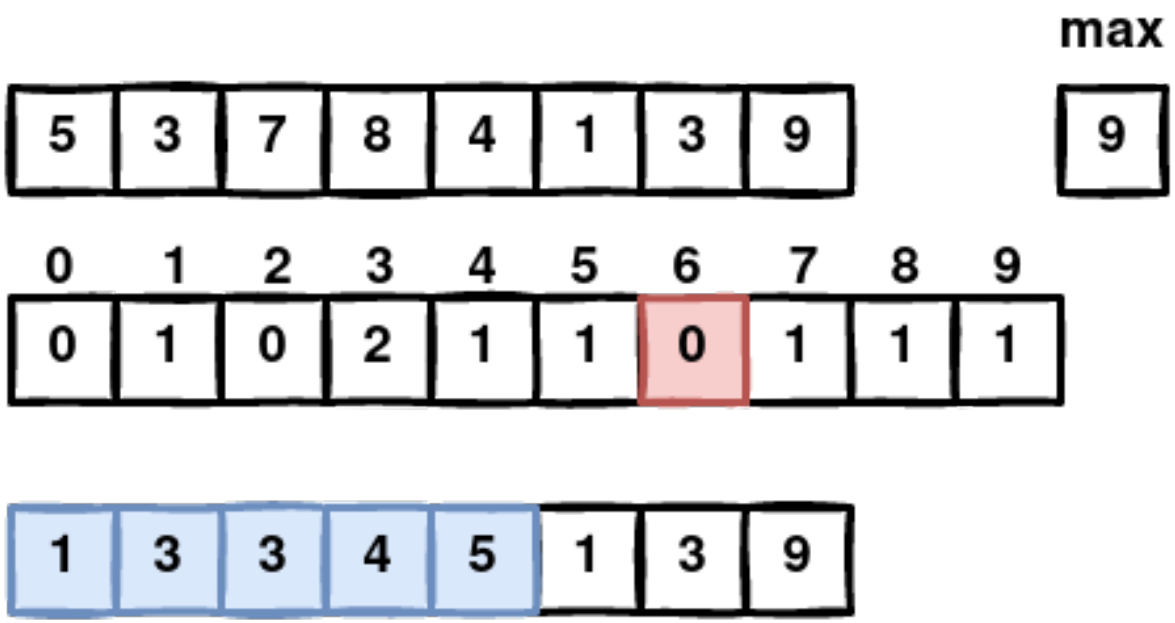
Counting Sort: Idea General



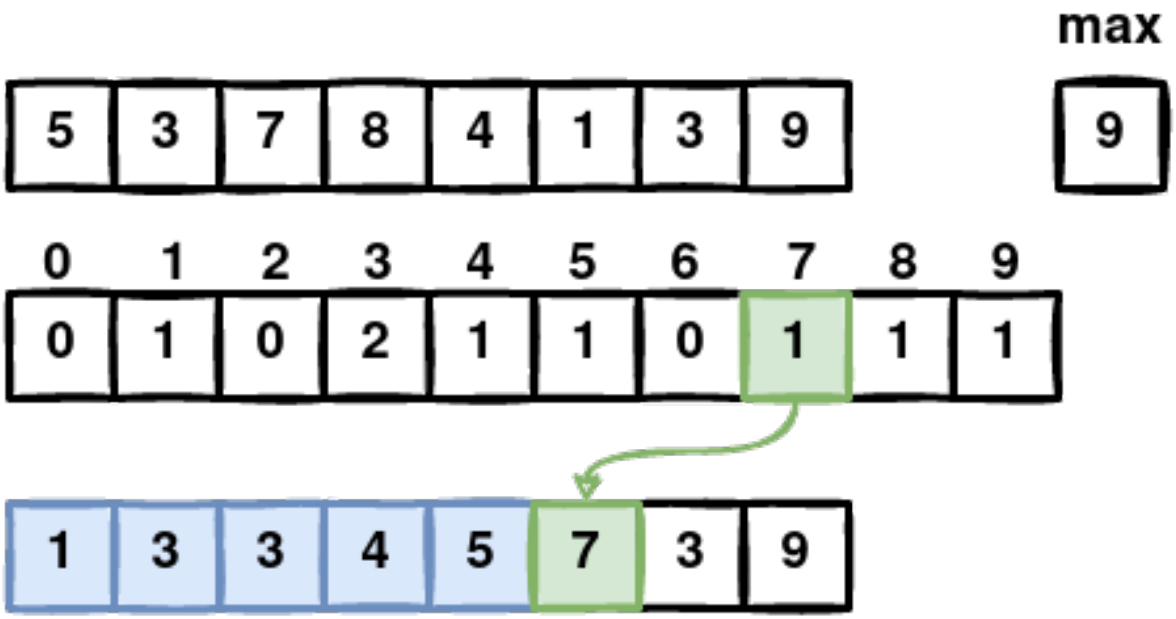
Counting Sort: Idea General



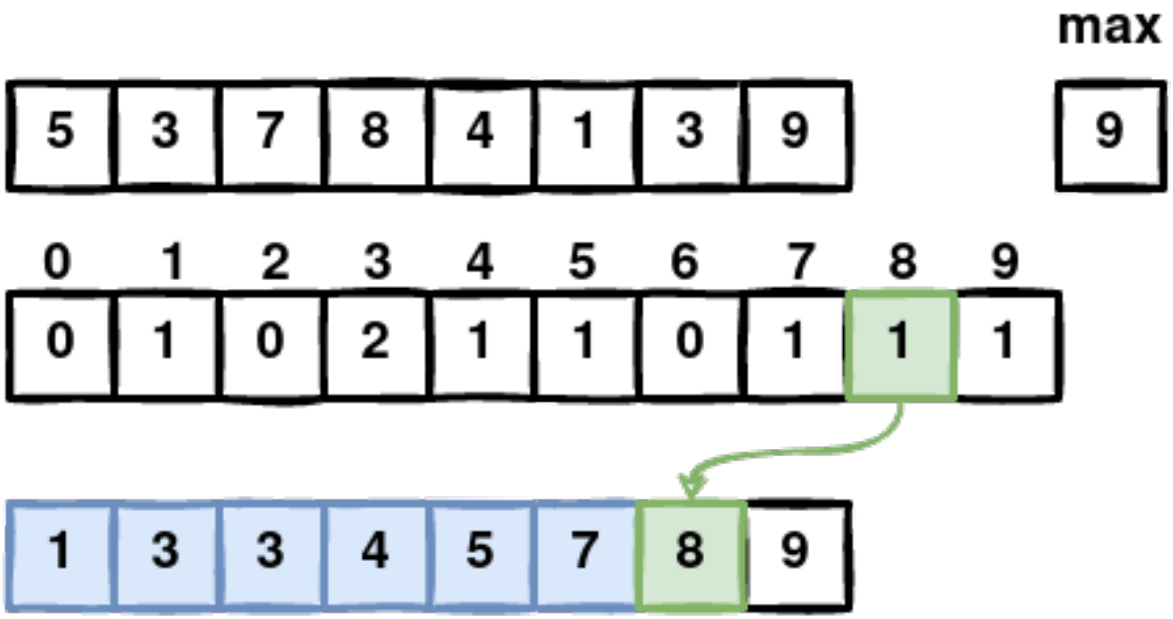
Counting Sort: Idea General



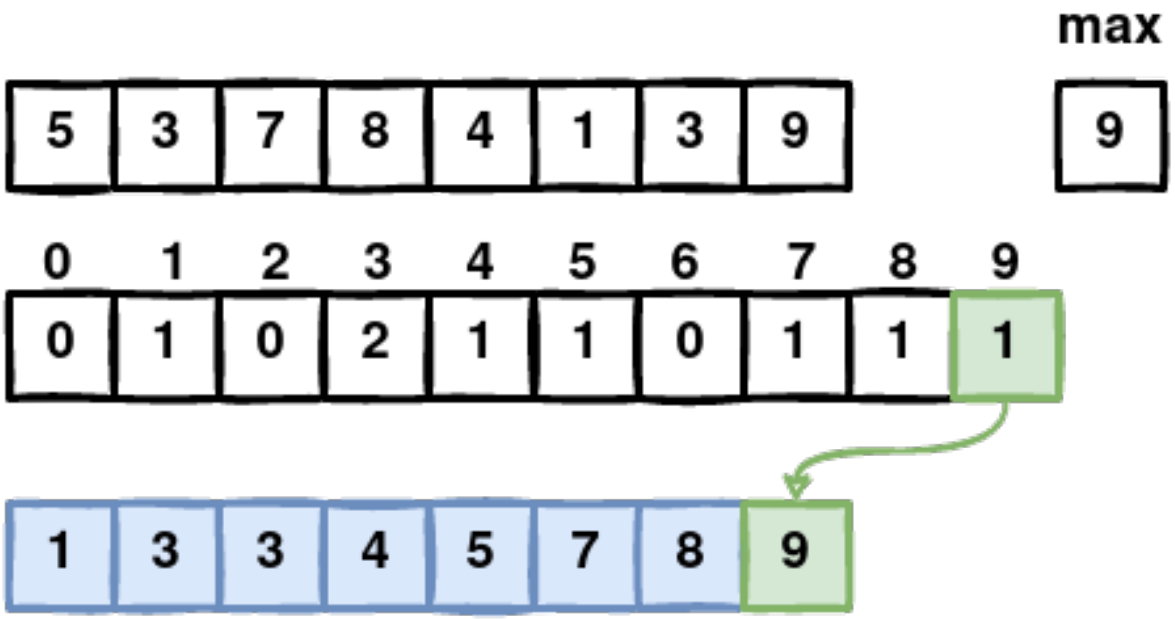
Counting Sort: Idea General



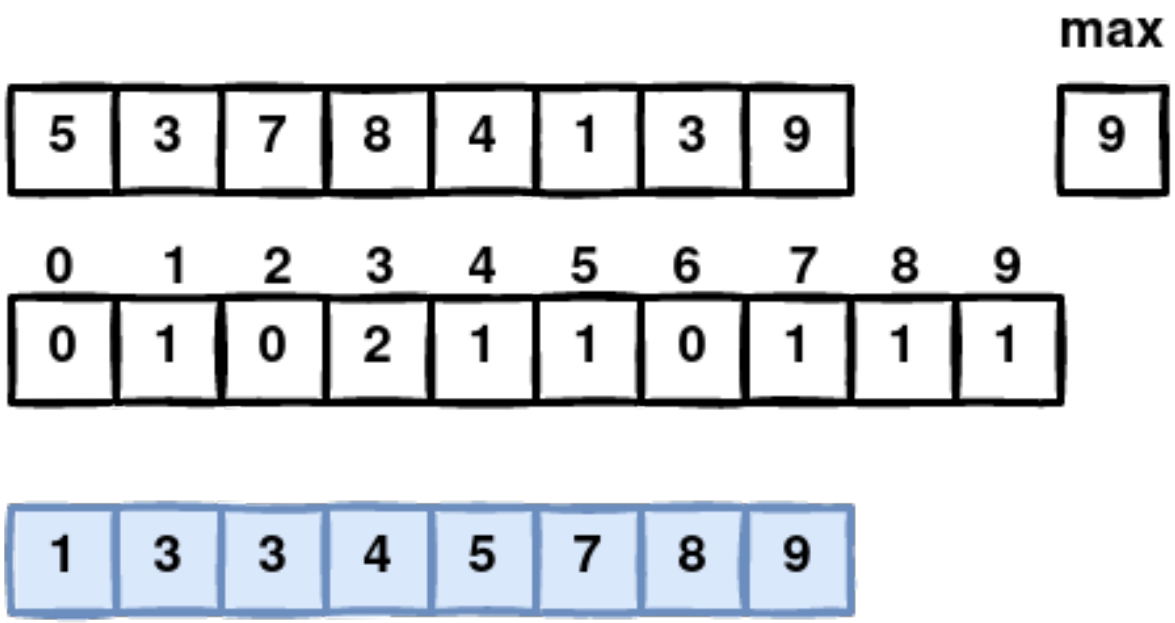
Counting Sort: Idea General



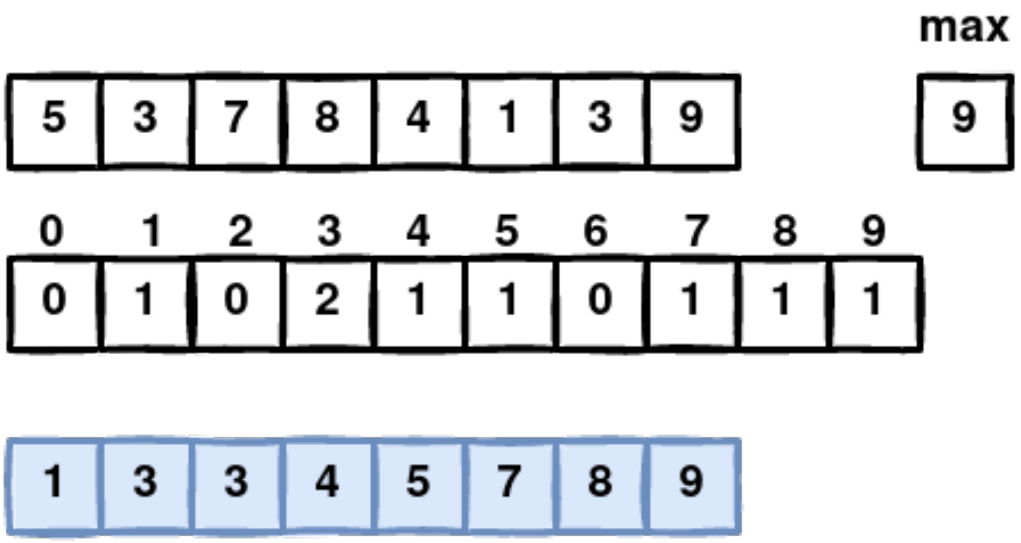
Counting Sort: Idea General



Counting Sort: Idea General



Counting Sort: Idea General



- En este primer acercamiento, el algoritmo depende fuertemente de k (máximo valor).
- Imaginense un arreglo de $n = 100$ elementos con máximo valor $k = 5$ millones.
- ¿Cómo podemos hacer que el ordenamiento dependa de n?
- ¿Cómo podemos diferenciar valores repetidos que se asocian a cosas distintas?

Counting Sort: Siguiente Índice

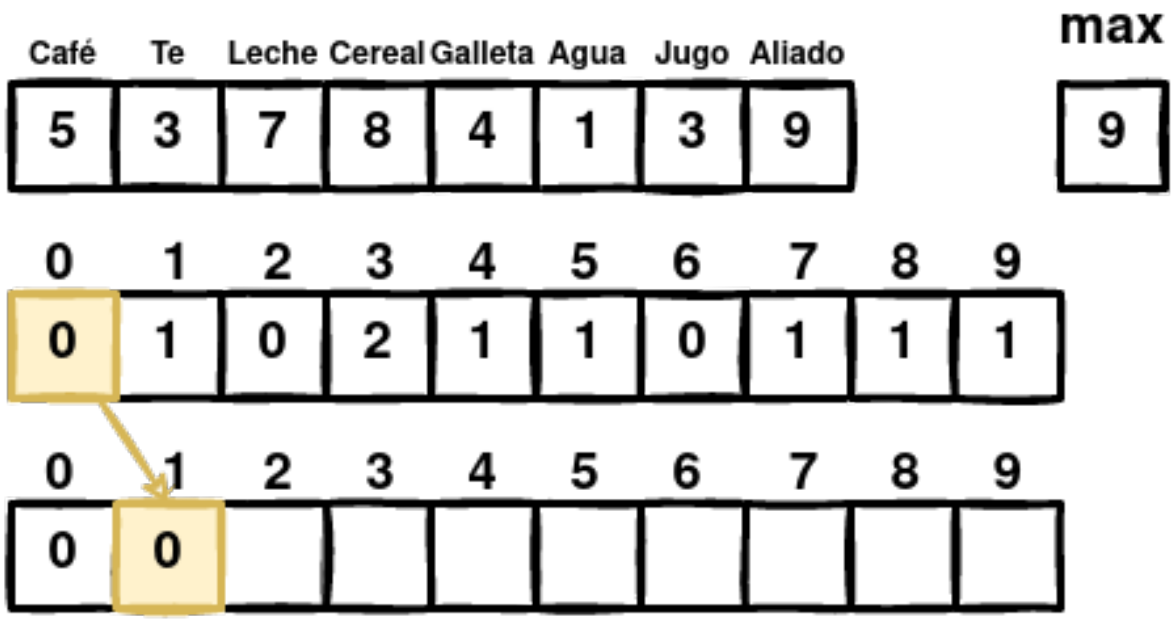
Café	Te	Leche	Cereal	Galleta	Agua	Jugo	Aliado
5	3	7	8	4	1	3	9

max: 9

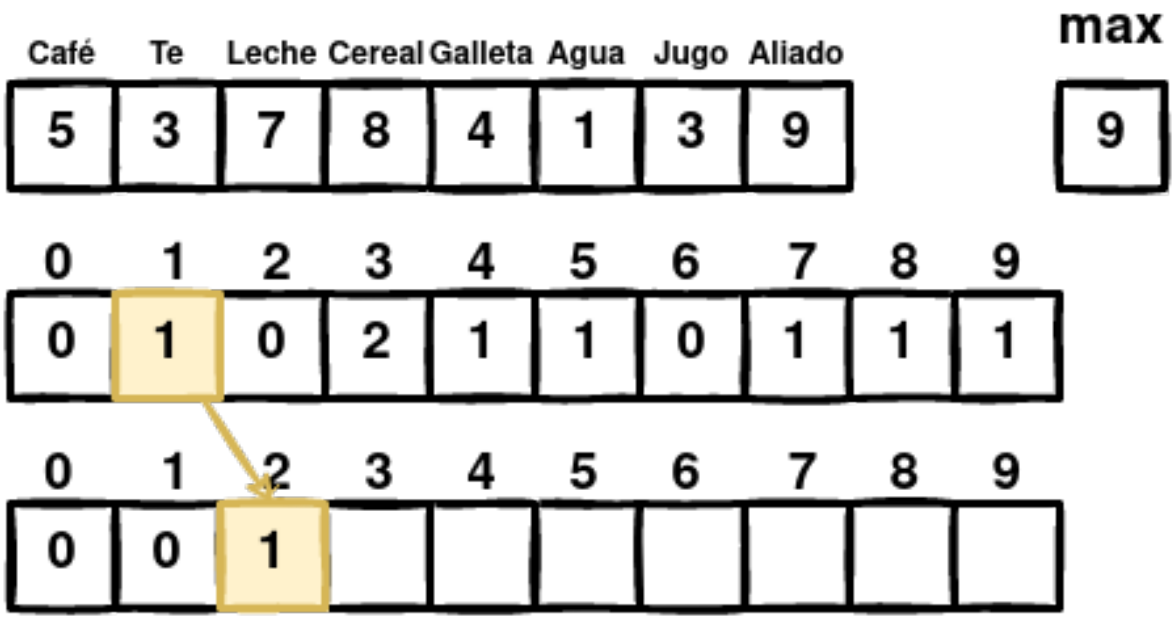
0	1	2	3	4	5	6	7	8	9
0	1	0	2	1	1	0	1	1	1

0	1	2	3	4	5	6	7	8	9
0									

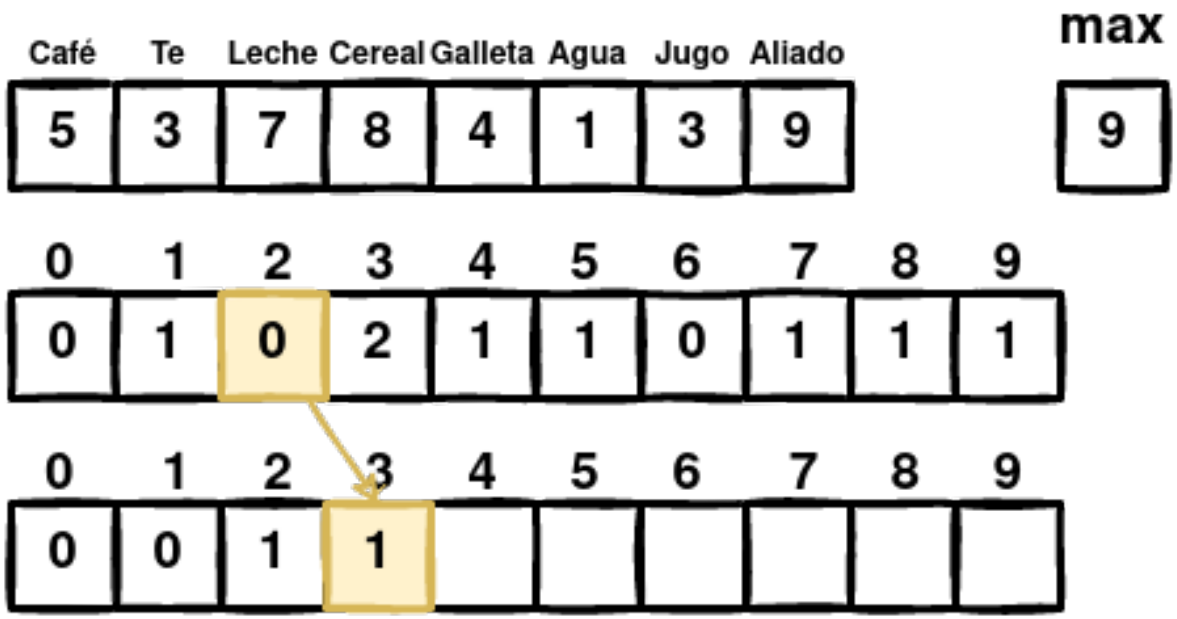
Counting Sort: Siguiente Índice



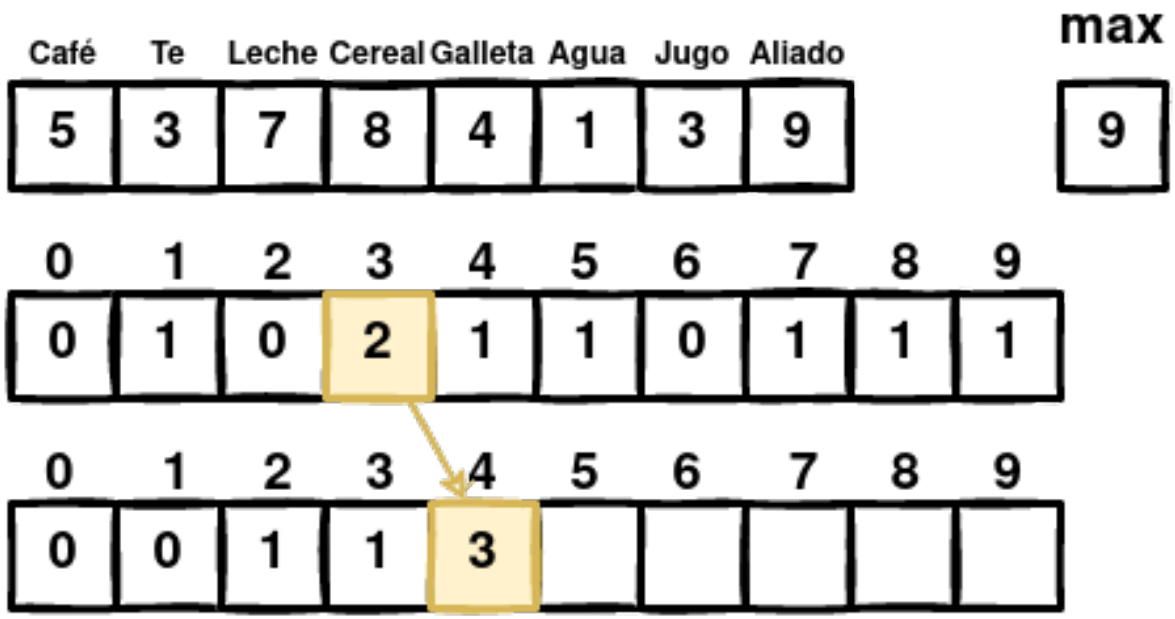
Counting Sort: Siguiente Índice



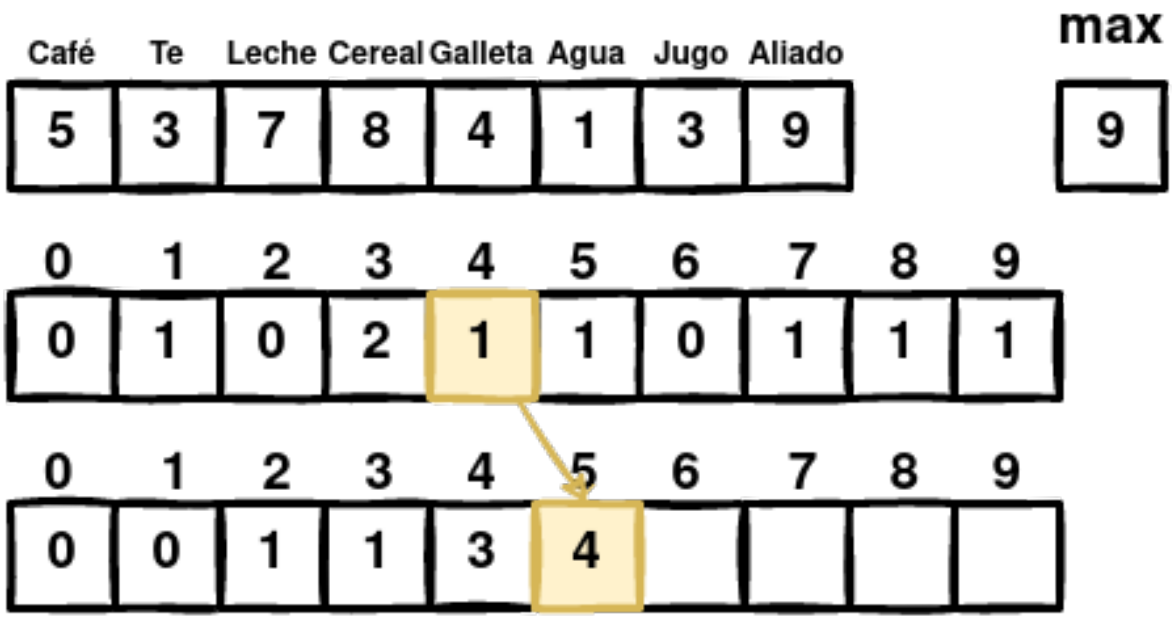
Counting Sort: Siguiente Índice



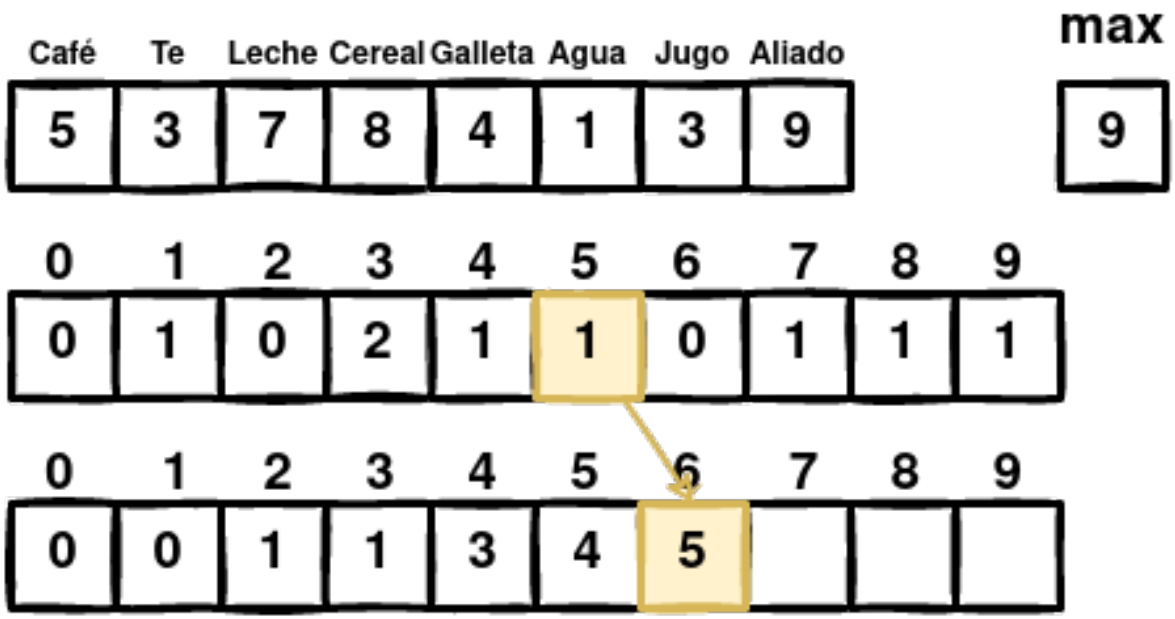
Counting Sort: Siguiente Índice



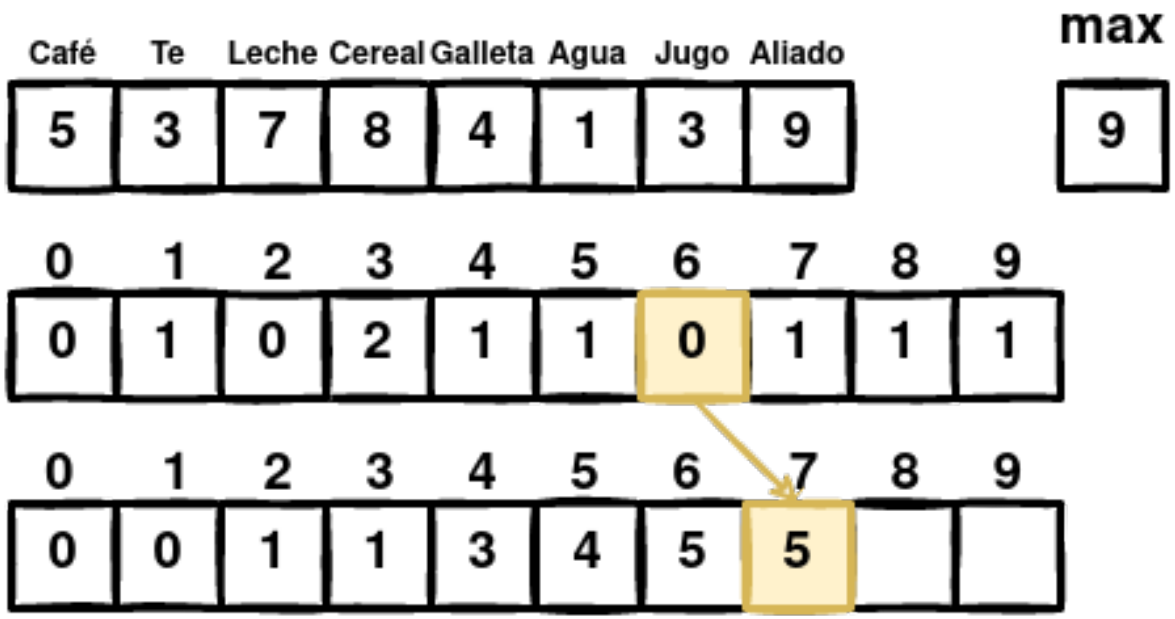
Counting Sort: Siguiente Índice



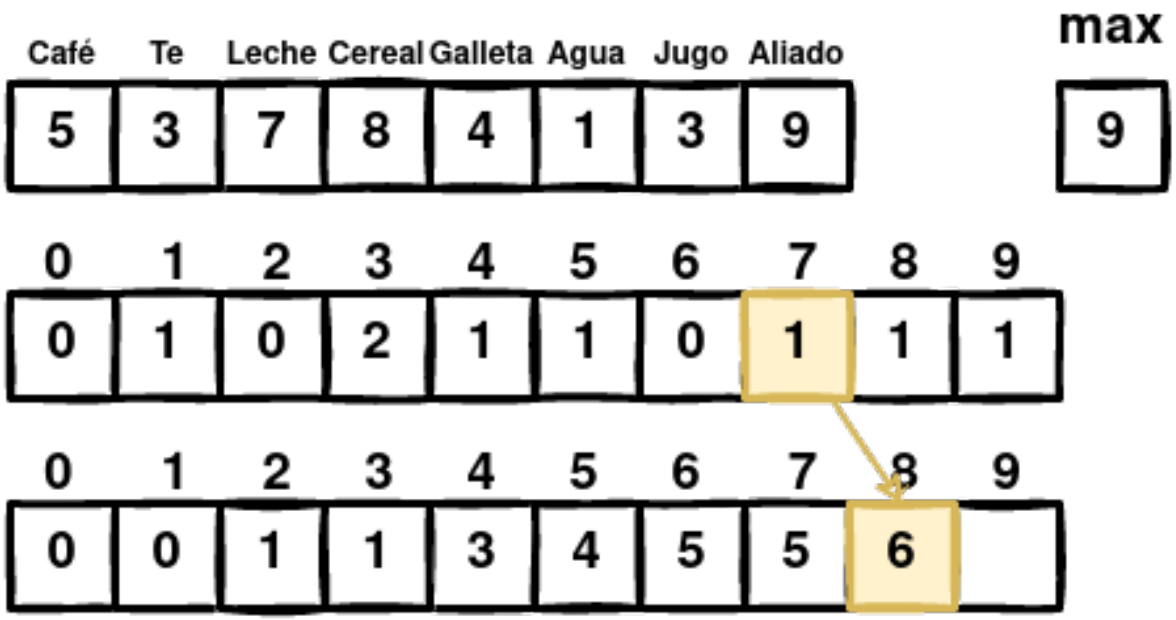
Counting Sort: Siguiente Índice



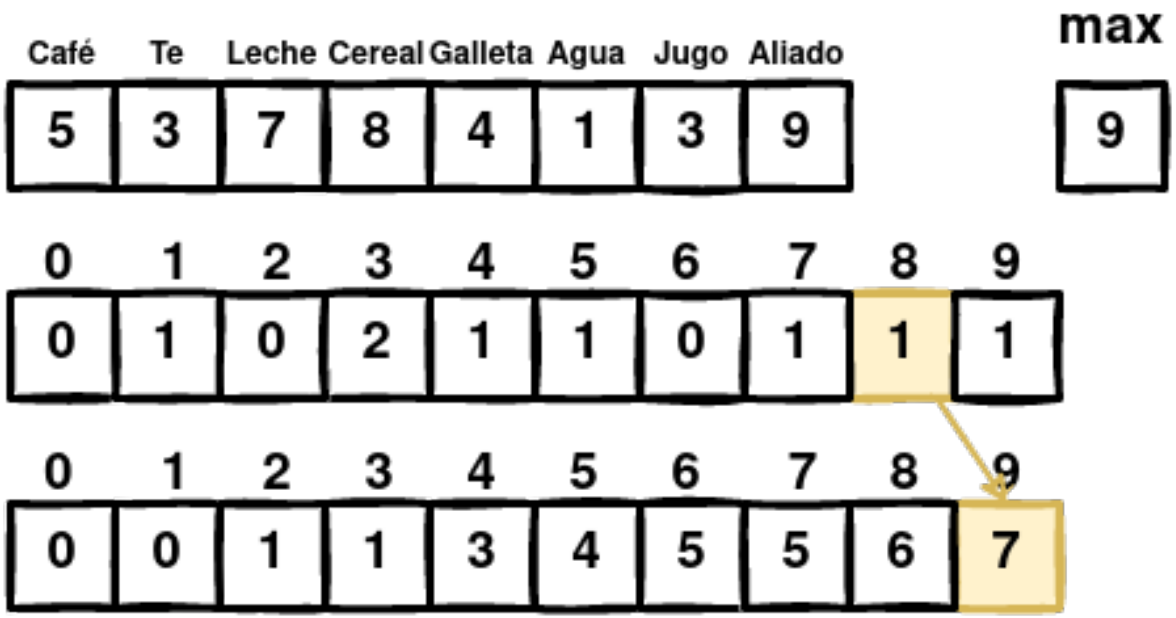
Counting Sort: Siguiente Índice



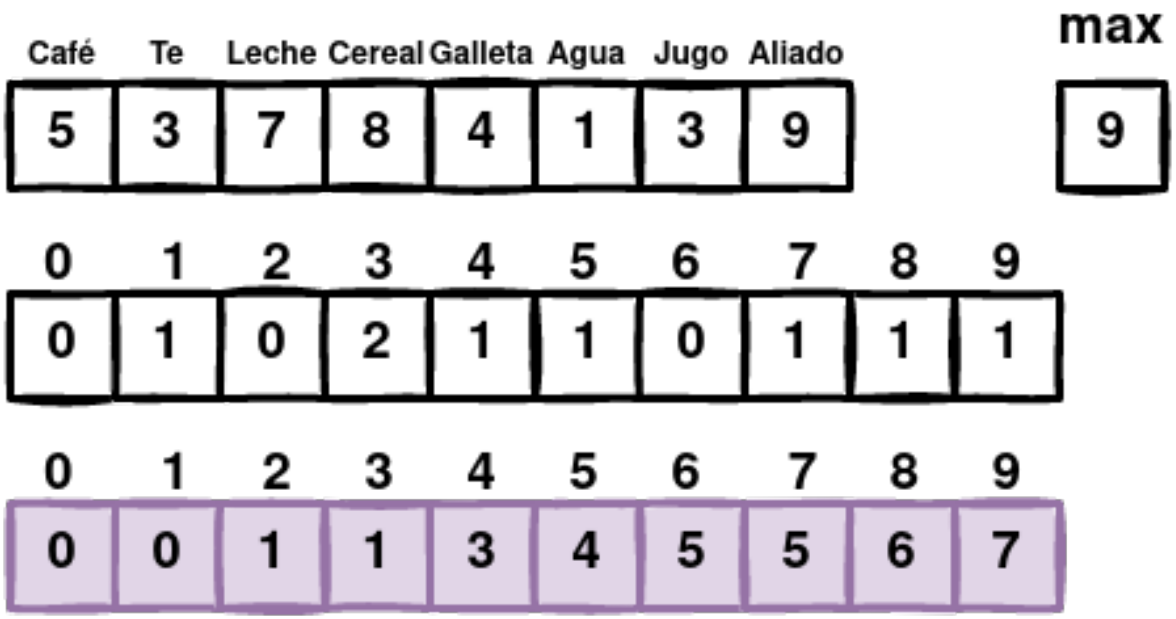
Counting Sort: Siguiente Índice



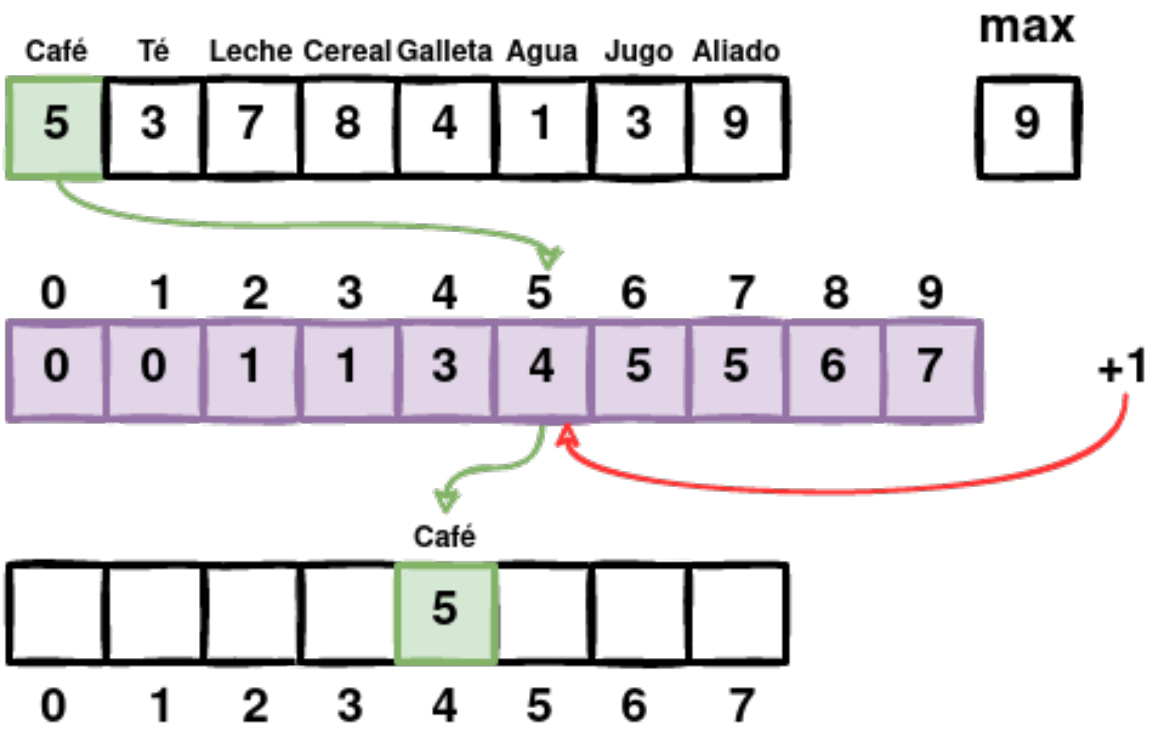
Counting Sort: Siguiente Índice



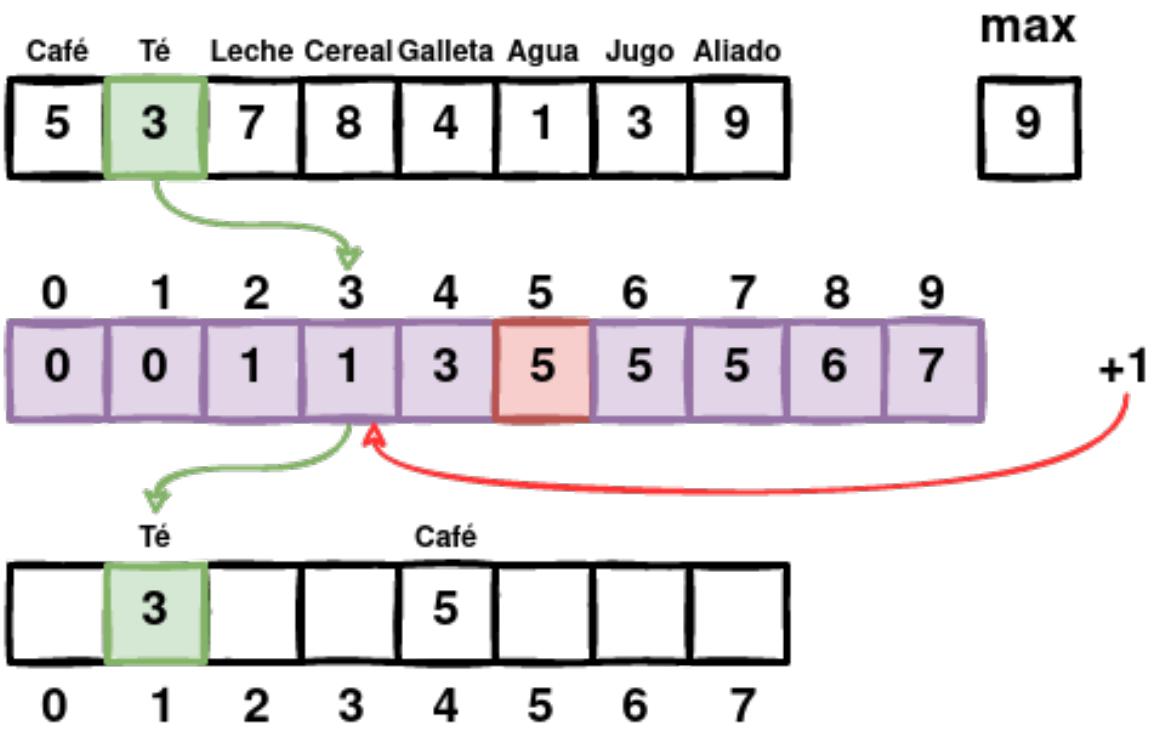
Counting Sort: Siguiente Índice



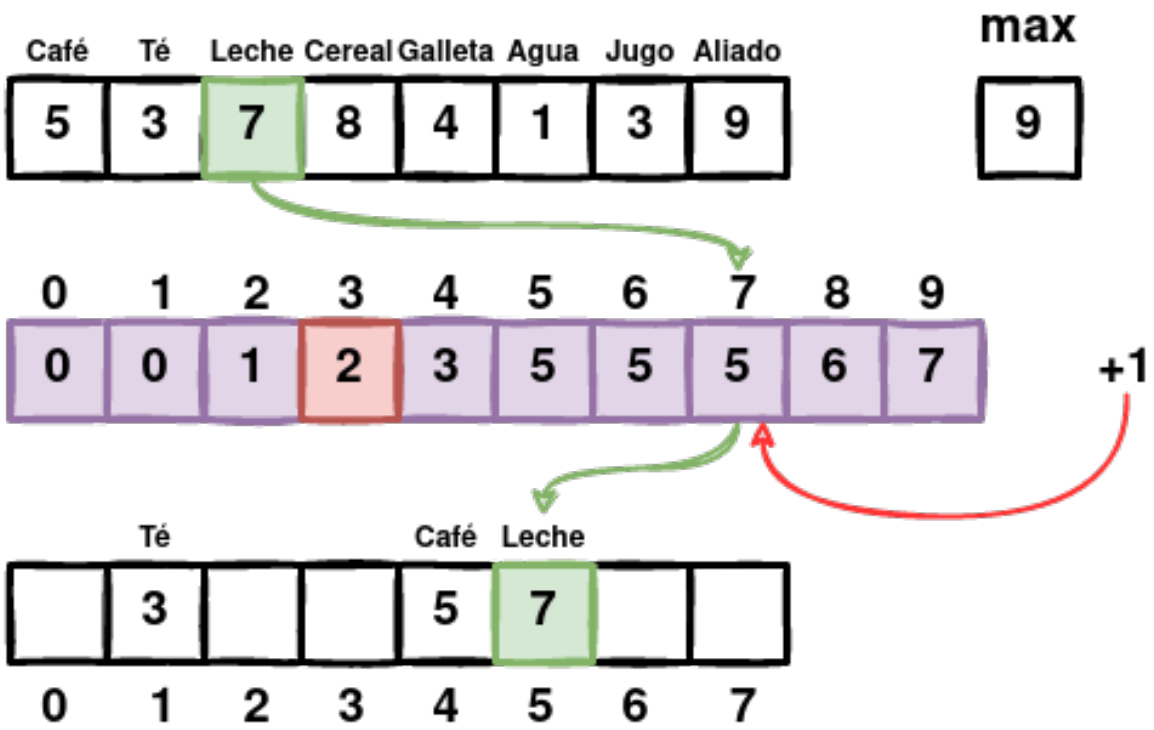
Counting Sort: Siguiente Índice



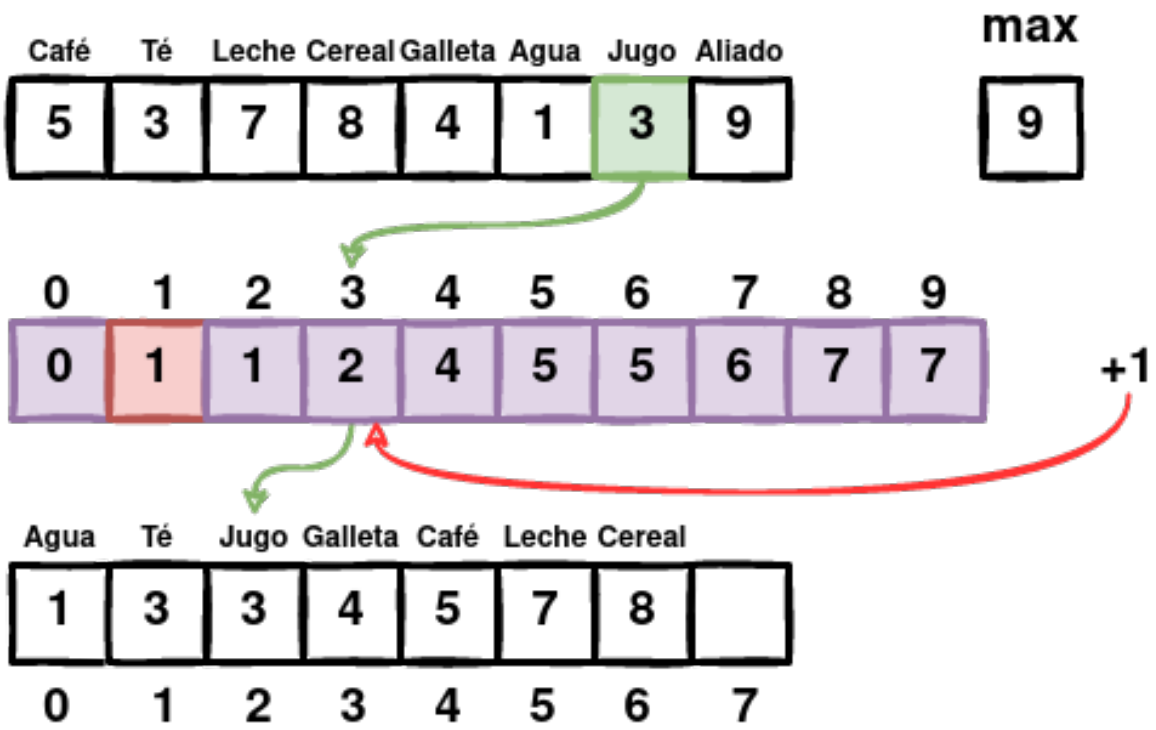
Counting Sort: Siguiente Índice



Counting Sort: Siguiente Índice



Counting Sort: Siguiente Índice



Counting Sort

Counting Sort

```
int * counting_sort(int A[], int max){
        int i, ind_sgte, veces;
        int largo = len(A); //funcion largo. ud la crean
        int B[largo]; //arreglo de salida.
        int C[max+1]; //arreglo contador

        for(i=0;i<=max;i++) //init arreglo
                C[i]=0;

        for(i=0;i<largo;i++)
                C[A[i]]++; //contando item de A es índice en C

        ind_sgte = 0;
        for(i=0;i<max+1;i++){ //indice siguiente
                veces=C[i];
                C[i] = ind_sgte;
                ind_sgte += veces;
        }

        for(i=0;i<largo;i++){ //paso a B
                B[C[A[i]]] = A[i];
                C[A[i]]++;
        }
        return B;
}
```

Pregunta

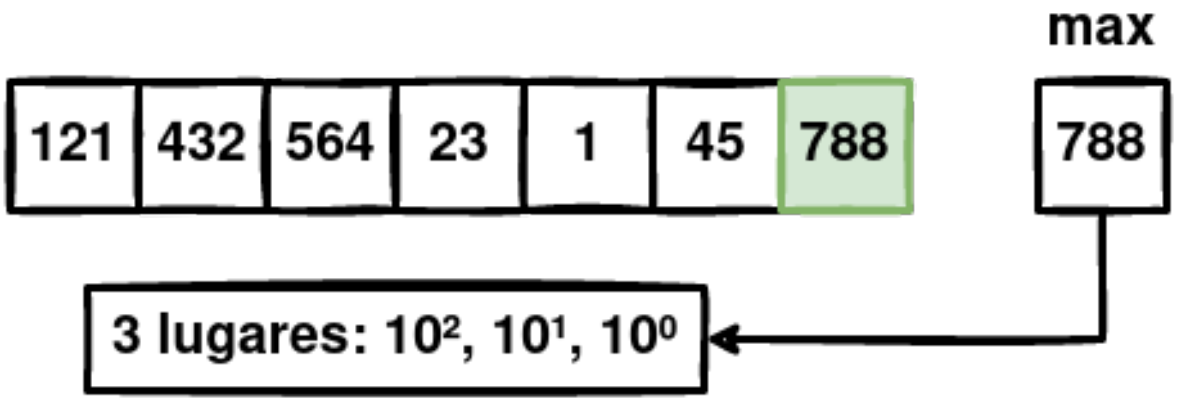
- Para ordenar enteros, ¿Es siempre mejor Counting Sort que algoritmos de Ordenamiento por Comparación?
- Si le piden ordenar un arreglo con numeros del 1 a n^3 ¿Usaría ud. Counting Sort como algoritmo? Justifique.

Radix Sort

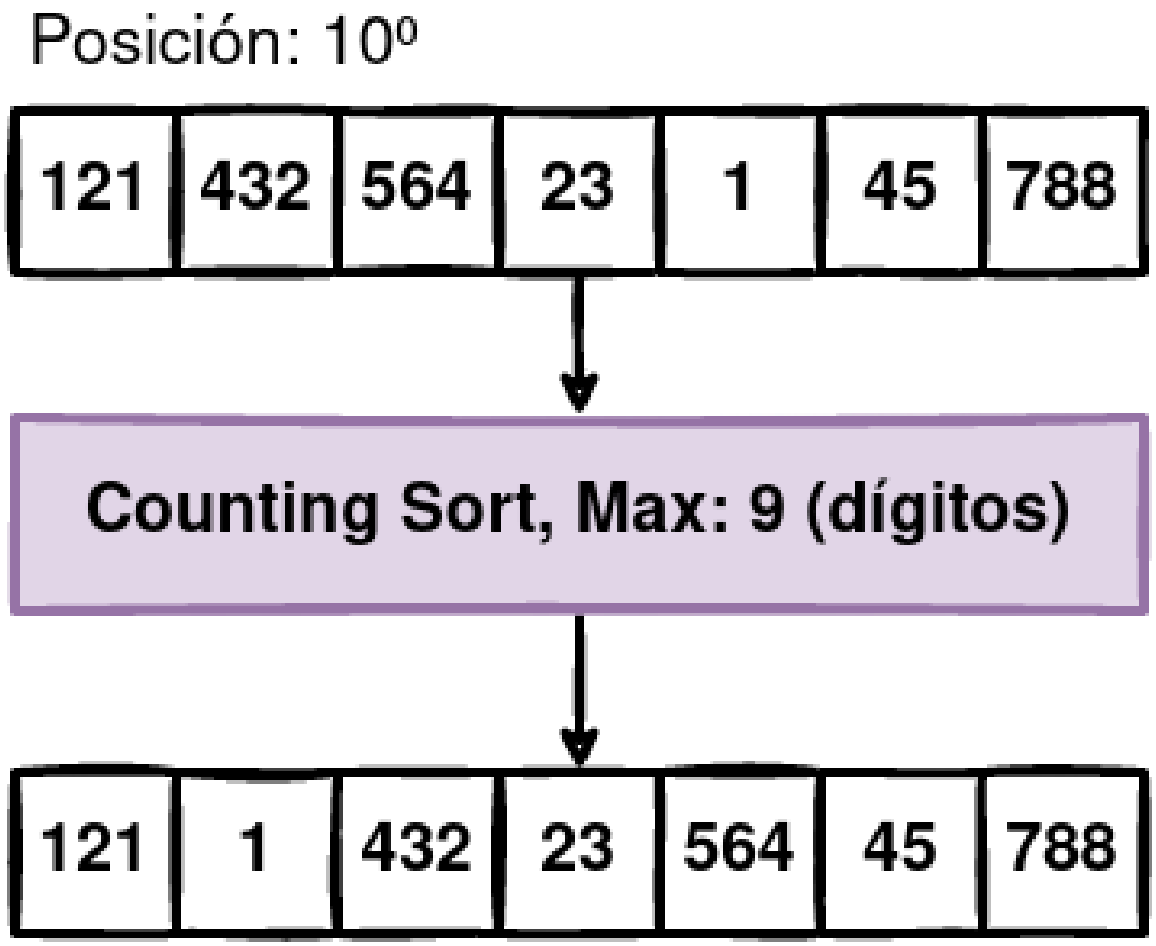
Algoritmo de Ordenamiento basado en dígitos, para poder ordenar desde las unidades hacia adelante, los elementos en un arreglo.

- Complejidad: $\mathcal{O}(d \times (n + k))$, con d la cantidad de digitos usados, n numero de elementos del arreglo y k el máximo elemento del arreglo.
- Suele ser usado para enteros, y representaciones discretas. ($k = 9$ =cte.)
- Por lo tanto, se suele decir que es $\mathcal{O}(d \times n)$.
- Se basa en una algoritmo estable para cada etapa $\rightarrow$ Counting Sort.
- Utiliza bastante memoria extra.

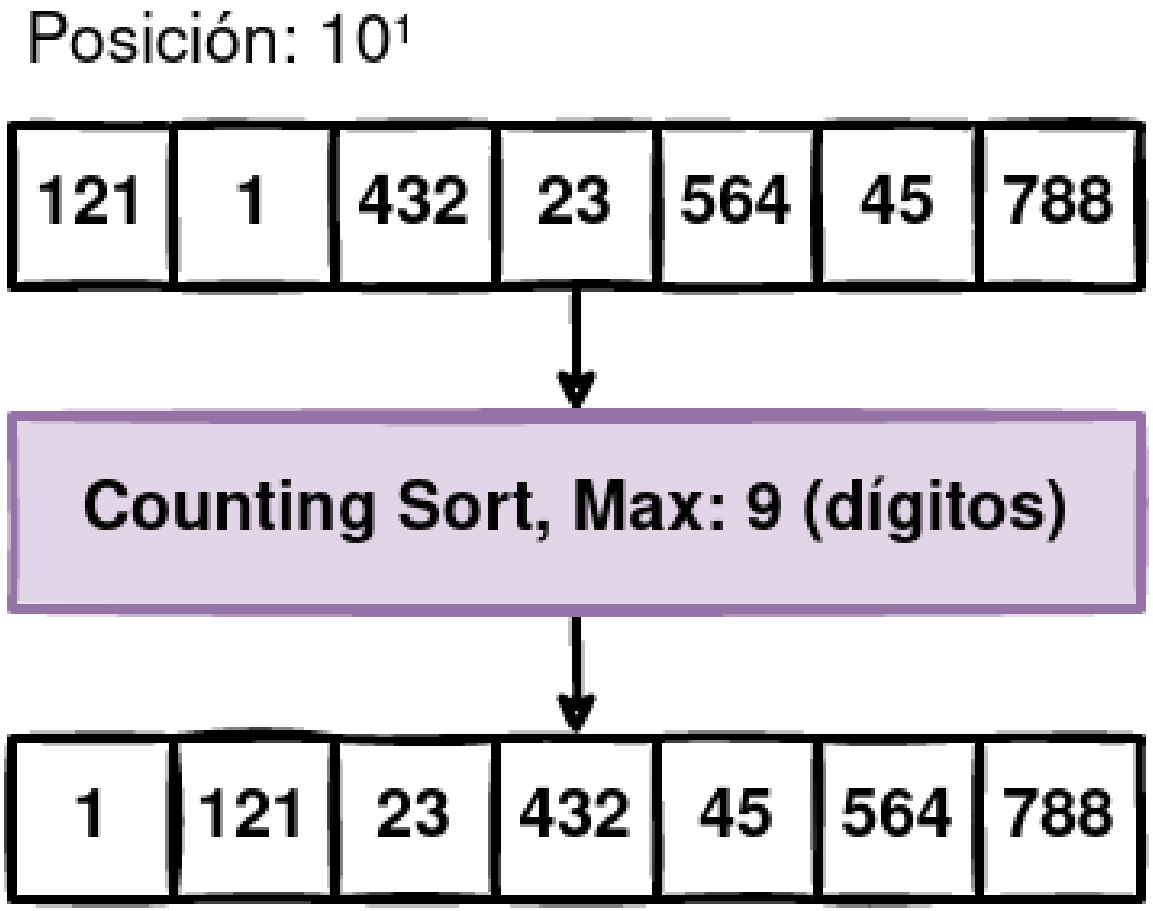
Radix Sort: Idea General



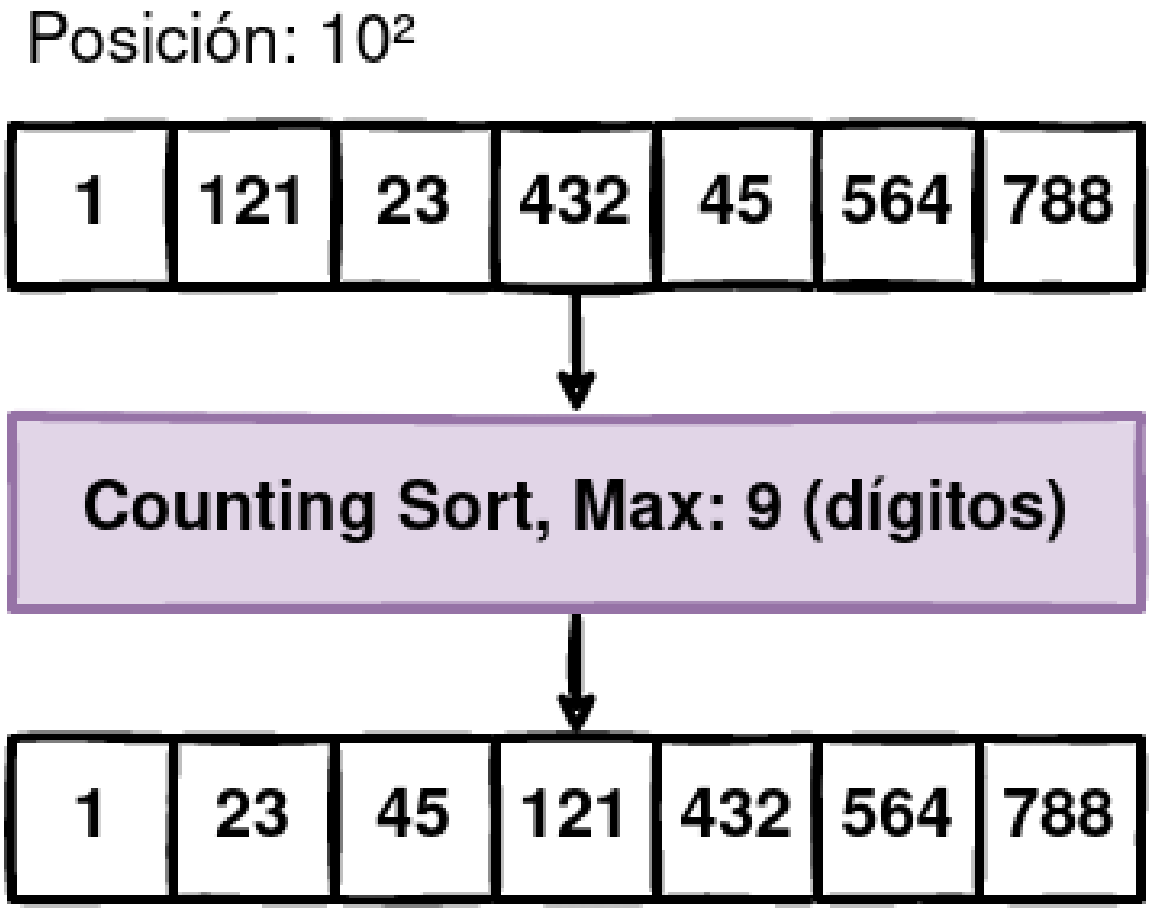
Radix Sort: Idea General



Radix Sort: Idea General



Radix Sort: Idea General



Radix Sort

Radix Sort

```
int maximo(int *A, int tam) {
        int i;
        int mayor = A[0];
        for (i = 1; i < tam; i++)
                if (A[i] > mayor)
                        mayor = A[i];
        return mayor;
}

void RadixSort(int *A, int tam) {
        int pos;
        int max = maximo(A, tam);

        for (pos=0 ; (max/ pow(10,pos)) > 0; pos++)
                CountingSort(A, tam, pow(10,pos));
}
```

Radix Sort

Radix Sort: built-in Counting Sort

```
void CountingSort(int *A, int tam, int lugar){
        int B[tam], i, ind_sgte, veces;
        int max = (A[0] / lugar) % 10; //por si no hay digitos finales
        for (i = 1; i < tam; i++) {
                if (((A[i] / lugar) % 10) > max)
                        max = (A[i]/lugar)% 10;
        }
        int C[max + 1];
        for (int i = 0; i < max; ++i)
                C[i] = 0;

        for (int i = 0; i < tam; i++)
                C[(A[i] / lugar) % 10]++;

        ind_sgte = 0;
        for(i=0;i<tam;i++){ //indice siguiente
                veces=C[i];
                C[i] = ind_sgte;
                ind_sgte += veces;
        }

        for(i=0;i<tam;i++){ //paso a B
                B[C[(A[i]/lugar) % 10]] = A[i];
                C[(A[i]/lugar) % 10]++;
        }
}
```

Ejercicios

- Programe Counting Sort de tal forma que agregue los elementos ordenados desde derecha a izquierda si es que estuviesen repetidos.
- Para ordenar enteros, ¿Es siempre mejor Counting Sort que algoritmos de Ordenamiento por Comparación?
- Si le piden ordenar un arreglo con numeros del 1 a n^3 ¿Usaría ud. Counting Sort como algoritmo? Justifique.

Ejercicios

- Utilizando RadixSort, ordene lexicográficamente la siguiente lista de palabras en inglés: cow, dog, sea, rug, row, mob, box, tab, bar, ear, tar, dig, big, tea, now, fox.